

Exercices Python — Jour 4 (partie 1) : chaînes de caractères et structures conditionnelles

Exercices — Python Jour 4 (Partie 1)

Méthodes de base des chaînes de caractères & structures conditionnelles

Contexte pédagogique : ces exercices portent **exclusivement** sur les notions vues dans le cours « *Chaînes de caractères et structures conditionnelles* » (Jour 4, Partie 1) :

- Méthodes `str` : `upper()`, `lower()`, `capitalize()`, `title()`, `strip()`, `lstrip()`, `rstrip()`, `in`, `find()`, `index()`, `count()`, `isdigit()`, `isalpha()`, `isalnum()`, `split()`, `join()`, `replace()`
- Structures conditionnelles : `if`, `elif`, `else`
- Opérateurs de comparaison : `==`, `!=`, `>`, `<`, `>=`, `<=`
- Opérateurs logiques : `and`, `or`, `not`
- Structure `match` / `case`
- La boucle `for` (déjà utilisée dans le cours pour parcourir le résultat de `split()`) et `len()` / `int()` (notions déjà acquises) restent autorisées.

Règle du jeu : interdiction d'utiliser des notions non vues (pas de fonctions `def`, pas de listes en compréhension, pas de dictionnaires, pas d'expressions régulières). Le but est de **réfléchir** à la bonne combinaison des outils ci-dessus, pas d'aller chercher des raccourcis plus avancés.

Consigne générale de rendu : pour chaque exercice, écrivez un script Python qui déclare vous-même les variables de test indiquées (pas besoin de `input()`), exécute le traitement demandé, et affiche des messages clairs avec `print()` / f-strings. Testez bien **toutes** les valeurs du jeu d'essai fourni : un bon script doit donner le bon résultat sur chacune d'entre elles.

Exercice 1



Normalisation et validation d'un identifiant utilisateur (login)

Contexte. Vous administrez un serveur d'authentification. Les utilisateurs saisissent leur identifiant (« login ») n'importe comment : espaces en trop, majuscules mélangées, caractères

interdits. Vous devez écrire un petit script qui **nettoie** puis **valide** chaque login avant de créer le compte.

Jeu d'essai. Testez votre script avec **chacune** de ces valeurs (une à la fois, ou dans une boucle **for** sur une liste) :

```
logins = [" KMabika ", "admin2024", " ", "J3an-Pierre!!", "root"]
```

Consignes.

1. Pour chaque login, supprimez les espaces superflus au début et à la fin.
2. Construisez la version normalisée du login en minuscules (c'est cette version qui sera enregistrée en base).
3. Un login est **valide** si :
 - il ne contient **que** des lettres et/ou des chiffres (pas de tiret, pas de !, pas d'espace restant, etc.) ;
 - **et** sa longueur est d'au moins 4 caractères après nettoyage.
4. Selon les cas, affichez avec **if / elif / else** l'un des trois messages suivants (un seul par login) :
 - "Login valide : <login_normalisé>"
 - "Login trop court"
 - "Login contient des caractères interdits"

Attention à l'ordre de vos tests : un login vide ou ne contenant que des espaces doit lui aussi être correctement rejeté.

Contraintes techniques obligatoires. `strip()`, `lower()`, `isalnum()`, `len()`, **if / elif / else**, au moins un opérateur logique (**and**, **or** ou **not**).

Résultat attendu (exemple).

```
" KMabika "      -> Login valide : kmabika
"admin2024"      -> Login valide : admin2024
" "             -> Login trop court
"J3an-Pierre!!" -> Login contient des caracteres interdits
"root"          -> Login trop court
```

Bonus (facultatif). Ajoutez un compteur qui affiche, à la fin, le nombre total de logins valides parmi le jeu de test.

Exercice 2



Validateur complet d'adresse IPv4

Contexte. Un script d'administration réseau reçoit des adresses IP saisies sous forme de texte (chaîne de caractères) et doit vérifier qu'elles sont **réellement valides** avant de les utiliser, en distinguant plusieurs types d'erreurs.

Jeu d'essai.

```
adresses = ["192.168.1.10", "10.0.0.256", "192.168.1", "abc.def.1.1", "
255.255.255.255", "01.02.03.04"]
```

Consignes. Pour chaque adresse de la liste :

1. Découpez la chaîne en morceaux à l'aide du caractère ..
2. **Vérifiez d'abord** que vous obtenez exactement 4 morceaux. Si ce n'est pas le cas, affichez : "<adresse> -> invalide : nombre d'octets incorrect (4 attendus)" et passez à l'adresse suivante.
3. Si vous avez bien 4 morceaux, vérifiez **pour chacun** qu'il ne contient que des chiffres. Si un seul morceau contient autre chose que des chiffres, affichez : "<adresse> -> invalide : caractère non numérique détecté".
4. Si tous les morceaux sont bien numériques, convertissez chacun en entier et vérifiez qu'il est compris entre 0 et 255 inclus. Si une valeur dépasse cette plage, affichez : "<adresse> -> invalide : valeur d'octet hors limites (0-255)".
5. Si toutes les vérifications précédentes sont passées avec succès, affichez "<adresse> -> adresse IP valide", puis affichez chaque octet sur une ligne séparée (comme dans le cours), au format " octet n°<position> : <valeur>".

Contraintes techniques obligatoires. `split()`, `isdigit()`, `len()`, boucle `for`, `if / elif / else`.

Bonus (facultatif).

1. À la fin du traitement de toute la liste, utilisez `join()` pour afficher un récapitulatif du type : "Bilan : 2 adresse(s) valide(s) sur 6 testées".
2. Réfléchissez : que se passe-t-il avec "01.02.03.04" dans votre script ? Est-elle acceptée ? Est-ce cohérent avec la définition d'une adresse IPv4 ? Ajoutez un commentaire expliquant votre choix.

Exercice 3



Analyseur de robustesse de mot de passe

Contexte. Avant de créer un compte administrateur, le système doit évaluer la robustesse du mot de passe choisi et classer chaque mot de passe en trois catégories : **Fort**, **Moyen** ou **Faible**.
Jeu d'essai.

```
mots_de_passe = ["azerty", "Azerty123", "P@sswOrd!", "12345678", "Ab1"]
```

Règles de classement. Un mot de passe est considéré comme :

- **Fort** si sa longueur est ≥ 10 , et qu'il contient **au moins une lettre, et au moins un chiffre, et au moins un caractère spécial** parmi !@#\$\$%^&*.
- **Moyen** si sa longueur est ≥ 10 , qu'il contient **au moins une lettre et au moins un chiffre**, mais **aucun** caractère spécial.
- **Faible** dans tous les autres cas.

Consignes.

1. Le mot de passe est une chaîne **mixte** (lettres + chiffres potentiellement) : vous ne pouvez donc pas appeler directement `isalpha()` ou `isdigit()` sur le mot de passe entier (il renverrait `False` alors qu'il contient bien des lettres ET des chiffres). Écrivez plutôt une boucle `for` qui parcourt **caractère par caractère** le mot de passe, et utilisez `isalpha()` / `isdigit()` sur **chaque caractère individuel** pour détecter la présence d'au moins une

lettre et d'au moins un chiffre.

2. Pour détecter un caractère spécial, parcourez la chaîne "!@#\$%^&*" avec une boucle `for` et l'opérateur `in` pour savoir si l'un de ces caractères apparaît dans le mot de passe (vous pouvez aussi vous aider de `count()`).
3. Une fois que vous avez déterminé, pour un mot de passe donné, trois informations (`contient_lettre`, `contient_chiffre`, `contient_special` — sous forme de `True/False`), calculez une variable `categorie` valant "Fort", "Moyen" ou "Faible" à l'aide d'un `if / elif / else` combinant `and/or/not` selon les règles ci-dessus.
4. Utilisez ensuite une structure `match categorie` pour afficher un message adapté :
 - "Fort" → "Mot de passe accepté : sécurité forte."
 - "Moyen" → "Mot de passe accepté mais amélioration conseillée (ajoutez un caractère spécial)."
 - "Faible" → "Mot de passe refusé : trop faible."
 - cas par défaut → "Statut inconnu."

Contraintes techniques obligatoires. boucle `for` caractère par caractère, `isalpha()`, `isdigit()`, `in`, `len()`, opérateurs logiques (`and`, `or`, `not`), `match / case`.

Résultat attendu (exemple).

```
azerty      -> Faible
Azerty123   -> Moyen
P@sswOrd!   -> Fort
12345678    -> Faible
Ab1         -> Faible
```

Bonus (facultatif). Ajoutez une règle supplémentaire : un mot de passe qui contient le mot "password" ou "azerty" (recherche insensible à la casse grâce à `lower()` et `in`) est automatiquement classé **Faible**, même s'il respecte les autres critères.

Exercice 4



Classificateur automatique de messages de logs

Contexte. Vous développez un petit outil qui lit des lignes de logs générées par un serveur et qui doit déterminer automatiquement leur **niveau de gravité**, afficher l'action à mener, et produire une version « archivable » du message.

Jeu d'essai.

```
logs = [
    "ERREUR: connexion au serveur impossible",
    "INFO: service démarre avec succes",
    "ATTENTION: espace disque faible",
    "ERREUR: ERREUR: double panne critique",
    "connexion etablie normalement",
]
```

Consignes. Pour chaque ligne de log :

1. Déterminez si le mot "ERREUR" apparaît, et si oui **combien de fois** (utilisez `count()`).
 2. Déterminez, à l'aide de l'opérateur `in`, si les mots "ATTENTION" ou "INFO" apparaissent également.
 3. Calculez une variable `niveau` (une chaîne de caractères) selon la logique suivante, en utilisant `if / elif / else` :
 - si "ERREUR" apparaît **2 fois ou plus** → `niveau = "CRITIQUE"`
 - sinon si "ERREUR" apparaît **exactement 1 fois** → `niveau = "ERREUR"`
 - sinon si "ATTENTION" apparaît → `niveau = "ATTENTION"`
 - sinon si "INFO" apparaît → `niveau = "INFO"`
 - sinon → `niveau = "INCONNU"`
 4. Utilisez ensuite `match niveau:` pour afficher l'action à prendre :
 - "CRITIQUE" → "Action : intervention immédiate requise !"
 - "ERREUR" → "Action : vérifier le service concerné."
 - "ATTENTION" → "Action : surveiller l'évolution."
 - "INFO" → "Action : aucune action nécessaire."
 - cas par défaut (`_`) → "Action : log à classer manuellement."
 5. Si le niveau est "CRITIQUE" ou "ERREUR", produisez une version corrigée du message avec `replace()` en remplaçant chaque occurrence de "ERREUR" par "RESOLU", puis affichez-la.
- Contraintes techniques obligatoires.** `count()`, `in`, `if / elif / else`, `match / case`, `replace()`.
- Bonus (facultatif).** À la fin de l'analyse de tous les logs, construisez avec `join()` une phrase récapitulative listant tous les niveaux rencontrés dans l'ordre, par exemple : "Résumé de la session : CRITIQUE - INFO - ATTENTION - CRITIQUE - INCONNU".

Exercice 5



Tableau de bord de supervision de services réseau

Contexte. Vous supervisez plusieurs services réseau (DNS, Web, Mail, DHCP, VPN, NTP...). Chaque service vous remonte son état sous forme de texte brut, pas toujours bien formaté. Vous devez construire un mini tableau de bord qui nettoie les données, affiche l'état de chaque service, et déclenche une alerte globale si nécessaire.

Jeu d'essai. Chaque élément de la liste est une chaîne au format "`nom:etat`" (notez les espaces mal placés, volontairement, sur certaines lignes) :

```
services = [
    "dns:UP",
    "web:DOWN",
    "mail:RESTARTING",
    "dhcp:UP",
    "vpn:DOWN",
    "  ntp : UP  ",
    "proxy:PANNE",
]
```

Consignes.

1. Pour chaque élément, séparez le nom du service et son état à l'aide de `split(":")`, puis nettoyez chacune des deux parties avec `strip()` (indispensable pour l'élément " ntp : UP ").
2. Normalisez le nom du service avec `title()` (pour un affichage soigné, ex. "Dns", "Ntp") et l'état avec `upper()` (pour être sûr de bien comparer, quelle que soit la casse d'origine).
3. À l'aide de `match etat;`, affichez pour chaque service un message :
 - "UP" → "<Nom> : le service répond correctement."
 - "DOWN" → "<Nom> : le service ne répond pas !"
 - "RESTARTING" → "<Nom> : le service est en cours de redémarrage."
 - cas par défaut (`_`) → "<Nom> : état non reconnu (<etat>)."
4. Pendant que vous parcourez la liste, comptez, à l'aide de comparaisons (`==`) et de compteurs que vous incrémentez vous-même dans le `if / elif / else` (ou après le `match`), le nombre de services "UP" et le nombre de services "DOWN".
5. À la toute fin, comparez les deux compteurs avec les opérateurs de comparaison et un opérateur logique si besoin, puis affichez :
 - si le nombre de services "DOWN" est **strictement supérieur** au nombre de services "UP" → "Alerte réseau : plus de services en panne qu'actifs !"
 - sinon si tous les services sont "UP" (aucun DOWN, aucun RESTARTING, aucun état inconnu) → "Réseau totalement opérationnel."
 - sinon → "Réseau globalement stable, surveillance recommandée."

Contraintes techniques obligatoires. `split()`, `strip()`, `title()`, `upper()`, `match / case`, opérateurs de comparaison, boucle `for`, au moins un compteur mis à jour manuellement.

Bonus (facultatif). Construisez, au fur et à mesure de la boucle, une liste Python `rapport = []` dans laquelle vous ajoutez (avec `.append(...)`, déjà vu précédemment) chaque ligne de message formatée, puis affichez le rapport complet en une seule fois à l'aide de `"\n".join(rapport)`.

Critères de réussite (auto-évaluation)

Avant de considérer un exercice comme terminé, vérifiez que :

- votre script fonctionne sur **toutes** les valeurs du jeu d'essai fourni, y compris les cas « pièges » (espaces, valeurs invalides, casse différente) ;
- vous n'utilisez **que** des notions vues dans le cours Jour 4 Partie 1 (pas de `def`, pas de dictionnaires, pas de regex) ;
- chaque méthode ou structure listée dans les « contraintes techniques obligatoires » de l'exercice apparaît bien au moins une fois dans votre code ;
- les messages affichés sont clairs et correspondent exactement à ce qui est demandé.